Universidad Autónoma de la Ciudad de México

Licenciatura en Ingeniería de Software

Arquitectura de computadoras

Práctica 03

Factorial Recursivo en Ensamblador x86

Alumno: Edson Joel Carrera Avila

Grupo: 302

Profesor: Prof. Mtro. Omar Nieto Crisóstomo

Fecha: April 20, 2026

Contents

1 Introducción **2**

2 Organización del programa **2**

3 Código fuente **2**

3.1 Programa principal (main.cpp) 2

3.2 Función ensambladora (factorial.asm) 3

4 Como funciona la recursión **4**

5 La Unidad de Punto Flotante (FPU) **5**

6 Ejemplo de ejecución **7**

7 Repositorio GitHub **7**

8 Conclusiones **7**

1 Introducción

Esta practica consiste en calcular el **factorial** de un numero usando dos lenguajes al mismo tiempo: **C++** (entrada y salida de datos) y **ensamblador x86** (operación matemática). La idea es ver como un programa de alto nivel puede delegar trabajo a código ensamblador, que esta mas cerca del hardware.

El factorial de un numero es multiplicar ese número por todos los enteros positivos menores que el. Por ejemplo:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120 \tag{1}$$

Matematicamente, se puede definir de forma **recursiva** (es decir, usando la misma operación pero con un valor mas pequeno cada vez):

$$x! = \begin{cases} 1 & \text{si } x \leq 1 \quad \text{(caso base)} \\ x \cdot (x-1)! & \text{si } x > 1 \quad \text{(paso recursivo)} \end{cases} \tag{2}$$

2 Organización del programa

El proyecto tiene dos archivos que trabajan juntos:

Table 1: Archivos del proyecto

Archivo	**Que hace**
`main.cpp`	Le pide un número al usuario, llama a la función ensambladora y muestra el resultado.
`Factorial.asm`	Contiene el calculo del factorial escrito en ensamblador.

El flujo del programa es sencillo:

1. El usuario escribe un número en la consola.
2. C++ recibe ese número y lo pasa a la función ensambladora.
3. La función ensambladora calcula el factorial de forma recursiva.
4. C++ toma el resultado y lo imprime en pantalla.

3 Código fuente

3.1 Programa principal (main.cpp)

El código en C++ hace tres cosas: declarar que la función `Mi_fact` existe en otro archivo, pedirle un numero al usuario, y mostrar el resultado.

```cpp
#include <iostream>
using namespace std;

// Le decimos a C++ que esta funcion esta definida en el archivo .asm
extern "C" double Mi_fact(double x);

int main() {
    double numero;
    cout << "--- Calculadora de Factorial (ASM Recursivo) ---" << endl;
    cout << "Introduce un numero: ";
    cin >> numero;

    double resultado = Mi_fact(numero);
    cout << "El factorial de " << numero
         << " es: " << resultado << endl;
    return 0;
}
```

Listing 1: main.cpp

La linea **extern "C"** es el puente entre C++ y ASM. Le dice al compilador que no modifique el nombre de la función al momento de compilar, para que ambos archivos puedan encontrarse.

3.2 Función ensambladora (factorial.asm)

Aqui esta el nucleo del programa. La función recibe `x`, comprueba si es 1 o menos (caso base), y si no, se llama a si misma con `x - 1` y multiplica el resultado por `x`.

```asm
.586
.model flat, c

.code

Mi_fact PROC

    ; Preparar el espacio en memoria para trabajar
    PUSH    EBP
    MOV     EBP, ESP
    SUB     ESP, 8

    FLD     QWORD PTR [EBP+8]      ; Cargar el valor x

    ; Comparar x con 1.0
    FLD1                           ; Cargar 1.0
    FCOMIP  ST(0), ST(1)           ; Comparar: 1.0 >= x?
    JAE     es_caso_base           ; Si x <= 1, ir al caso base

    ; x > 1: calcular x * factorial(x - 1)
    FLD1
    FSUBP   ST(1), ST(0)           ; Calcular x - 1
    FSTP    QWORD PTR [EBP-8]      ; Guardar x - 1 temporalmente

```

```
    PUSH    DWORD PTR [EBP-4]
    PUSH    DWORD PTR [EBP-8]
    CALL    Mi_fact                 ; Llamarse a si misma con (x - 1)
    ADD     ESP, 8

    FMUL    QWORD PTR [EBP+8]       ; Multiplicar resultado por x
    JMP     fin

es_caso_base:
    FSTP    ST(0)                   ; Descartar x
    FLD1                            ; Retornar 1.0

fin:
    MOV     ESP, EBP
    POP     EBP
    RET

Mi_fact ENDP
END
```

Listing 2: Factorial.asm

4 Como funciona la recursión

Cada vez que la función se llama a si misma, guarda su estado actual en la **pila de memoria** resuelve el subproblema mas pequeño, y al regresar retoma donde se quedo.

Para `Mi_fact(4.0)`, la cadena de llamadas es:

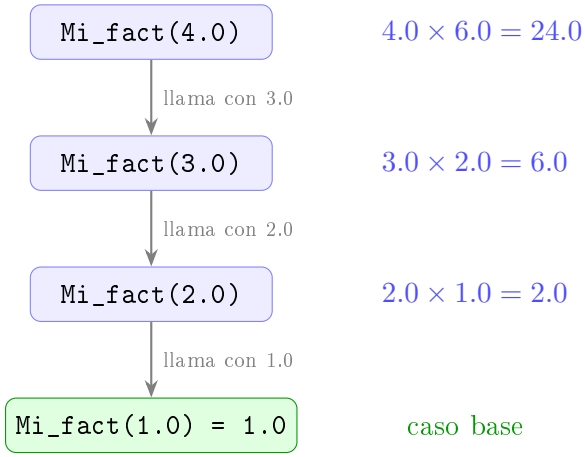

Figure 1: Cadena de llamadas recursivas para `Mi_fact(4.0) = 24`

Cada nivel espera a que el nivel inferior termine, multiplica el resultado por su `x` y lo regresa al nivel superior.

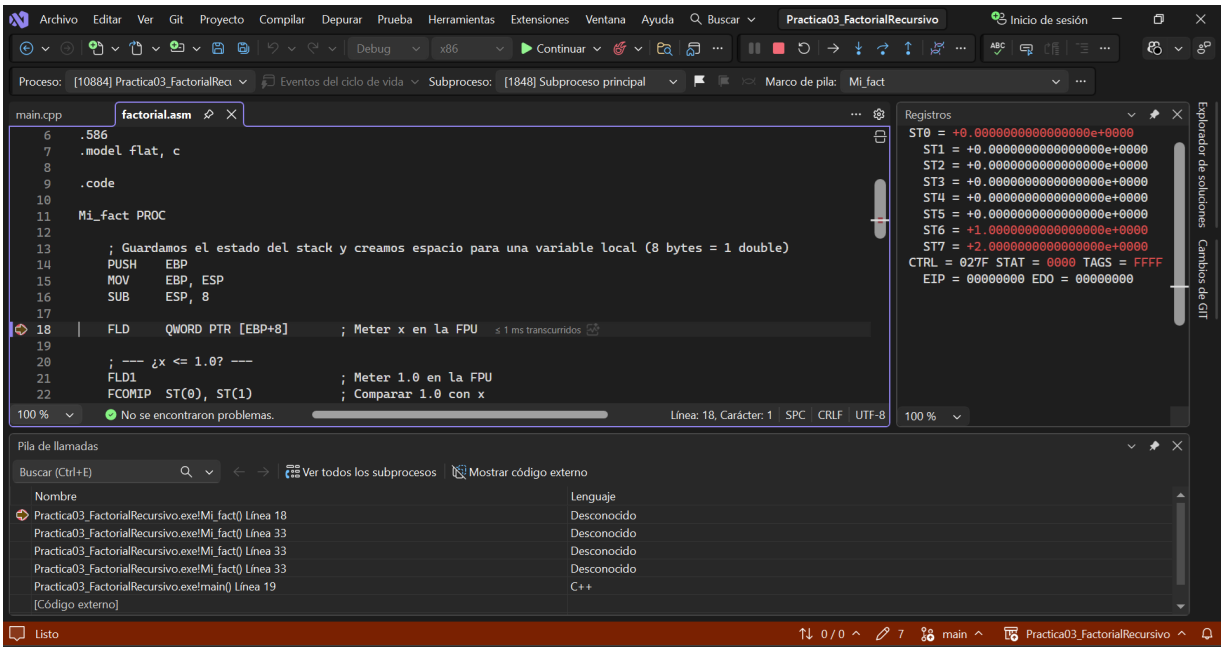

Figure 2: VS mostrando las llamadas apiladas de Mi_fact.

5 La Unidad de Punto Flotante (FPU)

El numero se maneja como `double` (numero con decimales de 64 bits) porque asi lo requiere la interfaz con C++. Para operar con este tipo de dato, el ensamblador usa la **FPU**, un componente del procesador dedicado a la aritmetica con decimales. La FPU tiene su propia pequeña pila interna de registros llamados `ST0` a `ST7`.

Las instrucciones FPU que usa esta practica son:

Table 2: Instrucciones FPU utilizadas

Instrucción	**Que hace en terminos simples**
`FLD`	Toma un numero de la memoria y lo coloca en la pila de la FPU.
`FLD1`	Coloca directamente el valor 1.0 en la pila FPU.
`FCOMIP`	Compara dos numeros de la pila y le avisa al procesador cual es mayor.
`FSUBP`	Resta los dos numeros del tope de la pila FPU.
`FSTP`	Saca el numero del tope de la pila FPU y lo guarda en memoria.
`FMUL`	Multiplica el tope de la pila por un numero en memoria.

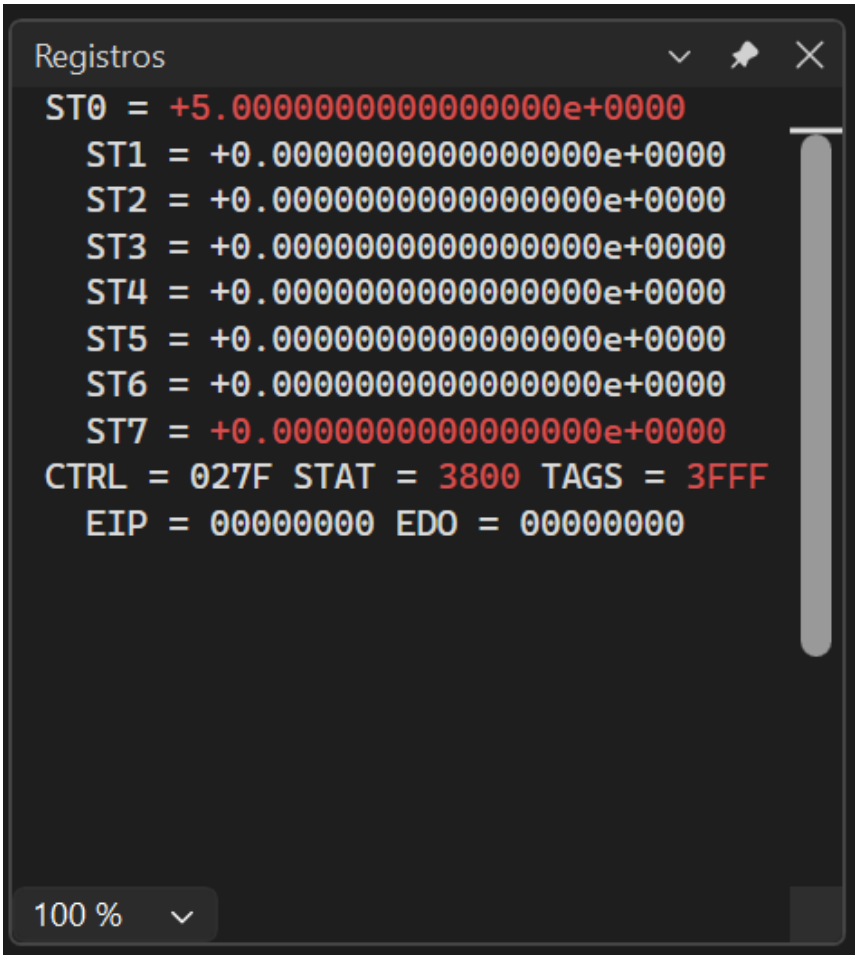

Figure 3: Ventana de registros con ST0-ST7 y sus valores númericos para 5!.

6 Ejemplo de ejecución

Al correr el programa, la consola muestra:

```
--- Calculadora de Factorial (ASM Recursivo) ---
Introduce un numero: 6
El factorial de 6 es: 720
```

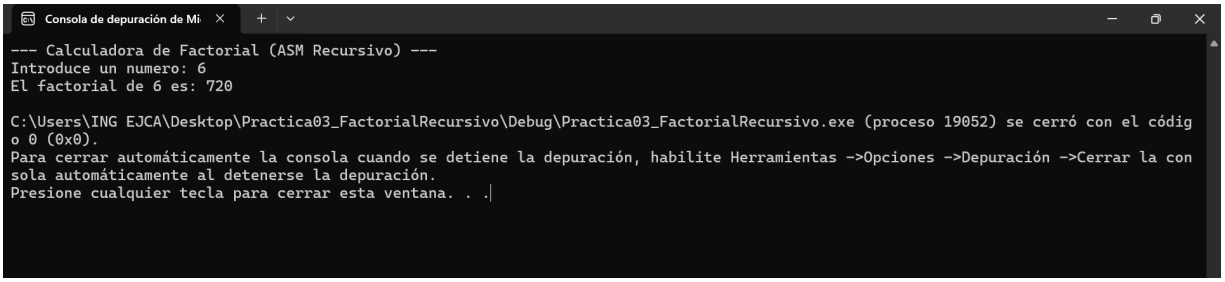

Figure 4: Vista de la consola, donde el usuario interactua.

7 Repositorio GitHub

https://github.com/7mo-ArquitecturaComputadoras/Practica03_FactorialRecursivo

8 Conclusiones

- Fue posible combinar C++ con ensamblador de forma sencilla gracias a `extern "C"`, que actua como puente entre los dos lenguajes.
- La recursión en ensamblador funciona igual que en cualquier lenguaje de alto nivel: cada llamada guarda su estado, espera el resultado del nivel inferior y continua. La diferencia es que en ensamblador todo ese manejo se hace instrucción por instrucción, de forma manual.
- El uso de la FPU permite trabajar con numeros de 64 bits, lo que hace a la función compatible con el tipo `double` de C++.
- Esta practica ayuda a entender lo que realmente ocurre dentro del procesador cada vez que un programa llama a una función, algo que los lenguajes de alto nivel ocultan automaticamente.